

بسمه تعالی

دکتر اثنی عشری

درس پردازش تکاملی

تمرین سری سوم

لطفا در حل تمرین به موارد زیر توجه داشته باشید:

- تمرین را خودتان انجام بدهید و به کاری که ارائه می کنید، تسلط کامل داشته باشید.
- تمرین را با هر زبان برنامه نویسی می توانید انجام دهید.
- به طور کلی، استفاده از کتابخانه های آماده مجاز نیست.
- اسم فایل ارسالی به صورت HW3_Name_StudentNumber باشد. همچنین اسم و شماره دانشجویی باید در فایل PDF پاسخ ها و داخل کدها به صورت comment وجود داشته باشد.
- فایل نهایی به صورت فشرده شده، باید شامل 1 فایل PDF (پاسخ سوالات و گزارش کدها) و یک فولدر شامل همه کدها (و فایل های لازم) باشد. از قرار دادن فایل های اضافی جدا خودداری فرمایید.
- گزارش باید کامل و در عین حال مختصر و مفید باشد. گزارش باید شامل توضیحات لازم (پاسخ سوالات پرسیده شده)، نکات پیاده سازی، مقادیر های پیرامون آنها و نتایج اجرای کد (به همراه مقایسه نتایج با هم در صورت نیاز) باشد.
- نمره هر تمرین از 3 بخش اصلی تشکیل می شود: گزارش، کد و ارائه تمرین. پس از تحویل هر تمرین، زمانی برای ارائه آن هماهنگ خواهد شد. تسلط شما بر موضوع و کد، بسیار مهم تر از صرف پاسخ دادن و پیاده سازی است که با پاسخ گویی به سوالات پرسیده شده در جلسه ارائه تمرین، سنجیده می شود.
- تمرین به صورت متنوع و گلچین آماده شده است تا با چالش های جدید و حتی مطرح نشده در کلاس روبرو شوید. هدف، افزایش قدرت تحلیل و حل مسئله شماست.
- در صورت مشاهده پاسخ ها و کدهای مشابه، نمره کسر می گردد.

سوال 1

الف) نرخ بازتولید (reproduction rate) چیست و زیاد یا کم بودن آن به چه معناست؟

ب) از دست دادن تنوع (loss of diversity) به چه عواملی بستگی دارد؟

ج) شدت انتخاب (selection intensity) چیست و به چه عواملی بستگی دارد؟

سوال 2) طراحی الگوریتم تکاملی مقاوم در برابر همگرایی زودهنگام

فرض کنید قصد دارید با استفاده از یک الگوریتم تکاملی، یک مسئله‌ی چندقله‌ای (Multimodal) را حل کنید؛ یعنی مسئله‌ای که چندین نقطه‌ی بهینه‌ی محلی در فضای جستجوی آن وجود دارد. در اجرای الگوریتم مشاهده می‌کنید که با وجود آغاز خوب، پس از چند نسل، همه‌ی جمعیت به تدریج روی یک ناحیه خاص از فضای جستجو متمرکز شده و دیگر نواحی با پتانسیل بالا کشف نمی‌شوند.

الف) اهمیت حفظ تنوع ژنتیکی در جمعیت را در مسائل چندقله‌ای شرح دهید. توضیح دهید که چگونه کاهش تنوع می‌تواند الگوریتم را در یک نقطه‌ی بهینه‌ی فرعی نگه دارد و چرا تنوع برای کشف نواحی دیگر در فضای جستجو ضروری است. همچنین، رابطه‌ی بین حفظ تنوع، نرخ اکتشاف (exploration) و نرخ بهره‌برداری (exploitation) را توضیح دهید.

ب) دو روش به اشتراک‌گذاری شایستگی (Fitness Sharing) و شلوغی (Crowding) برای حفظ تنوع در الگوریتم‌های تکاملی را به صورت مفهومی شرح دهید. برای هر روش، به این پرسش‌ها پاسخ دهید:

- منطق اصلی عملکرد آن چیست؟
- در چه شرایطی مؤثرتر است؟
- برای استفاده از این روش چه پارامترهایی باید تنظیم شوند؟
- آیا اجرای آن برای جمعیت‌های بزرگ محاسباتی مقرون به صرفه است؟

- این روش چگونه از تمرکز بیش از حد جمعیت در یک ناحیه جلوگیری می کند؟
 - صریح یا ضمنی (explicit / implicit) بودن هر روش را مشخص کنید و مقایسه ای بین نقاط قوت و ضعف آن ها ارائه دهید.
 - پ) اگر بخواهید یک الگوریتم تکاملی طراحی کنید که بتواند در یک فضای چندقله ای به درستی عمل کند و از همگرایی زود هنگام جلوگیری کند، چه انتخاب هایی خواهید داشت؟ به موارد زیر اشاره کنید:
 - از چه روش انتخابی (selection method) استفاده می کنید و چرا؟ به ویژه توضیح دهید که چگونه انتخاب پارامترهایی مانند اندازه ی تورنومنت یا فاکتور فشار در ranking بر رفتار الگوریتم تأثیر می گذارد.
 - از کدام مکانیزم حفظ تنوع استفاده می کنید؟ چرا آن روش را انتخاب کرده اید؟ چگونه پارامترهای آن را تنظیم می کنید؟
 - اگر تصمیم دارید ساختار الگوریتم را به صورت چندجمعیتی (multi-population) طراحی کنید، نحوه ی تقسیم جمعیت، مهاجرت بین زیرجمعیت ها و زمان بندی آن را نیز توضیح دهید.
- تحلیل شما باید توجیه علمی داشته باشد و هدف آن رسیدن به تعادلی مؤثر میان اکتشاف، بهره برداری، و پایداری بلندمدت باشد.

سوال 3) پیاده‌سازی: حل مساله سودوکو با استفاده از الگوریتم ژنتیک

مساله سودوکو یک بازی پازل منطقی محبوب است که شامل یک ماتریس 9×9 می‌باشد. هدف در این مساله، پر کردن تمام خانه‌های خالی جدول با اعداد ۱ تا ۹ است به گونه‌ای که در هر سطر، هر ستون، و هر زیرماتریس 3×3 هیچ عدد تکراری وجود نداشته باشد. برخی از خانه‌ها در ابتدای مساله مقدار دارند و قابل تغییر نیستند. این مساله در دسته مسائل رضایت محدودیت (CSP) قرار می‌گیرد و به عنوان یک مساله بهینه‌سازی نیز می‌تواند مدل شود. در این پروژه، هدف ما استفاده از الگوریتم ژنتیک به عنوان یک روش فراابتکاری برای یافتن راه‌حل معتبر سودوکو است. الگوریتم ژنتیک با الهام از فرآیند انتخاب طبیعی عمل می‌کند و مجموعه‌ای از جواب‌های ممکن (جمعیت) را به تدریج با ترکیب، جهش و انتخاب بهبود می‌بخشد.

در این روش، هر فرد (کروموزوم) یک جدول کامل سودوکو است که در آن خانه‌های ثابت بدون تغییر باقی می‌مانند و بقیه خانه‌ها با اعداد ۱ تا ۹ مقداردهی می‌شوند. سطرها و ستون‌ها به گونه‌ای مقداردهی می‌شوند که درون هر سطر تکراری نباشد. این کار باعث کاهش فضاها نامعتبر می‌شود.

تابع شایستگی یکی از اجزای کلیدی الگوریتم ژنتیک در این پروژه است. این تابع، برای هر جدول سودوکو تولیدشده، میزان اعتبار آن را بر اساس میزان رعایت قوانین سودوکو اندازه‌گیری می‌کند. به‌طور خاص، در این تابع تعداد اعداد یکتا در هر ستون و در هر بلوک 3×3 محاسبه می‌شود و مجموع آن‌ها به عنوان امتیاز شایستگی در نظر گرفته می‌شود. اگر در هر ستون و هر بلوک ۹ عدد یکتا وجود داشته باشد، مقدار شایستگی حداکثر یعنی ۱۶۲ خواهد بود ($9 \times 9 + 9 \times 9$). هر چه این عدد به ۱۶۲ نزدیک‌تر باشد، جدول معتبرتر و فرد مورد نظر بهتر است.

جمعیت اولیه به صورت تصادفی ایجاد می‌شود، به طوری که مقادیر داده شده حفظ می‌شوند و خانه‌های باقی‌مانده در هر سطر با اعداد تصادفی بدون تکرار مقداردهی می‌شوند.

برای پیاده‌سازی الگوریتم ژنتیک در این پروژه، پیشنهاد می‌شود از روش انتخاب Rank-based Selection یا Stochastic Universal Sampling (SUS)، از روش تقاطع تک‌نقطه‌ای یا یکنواخت، و از روش جهش مبتنی بر جابجایی استفاده شود. انتخاب روش‌های مناسب در هر مرحله می‌تواند با توجه به اهداف، پیچیدگی پیاده‌سازی، و میزان تنوع موردنیاز انجام شود.

در بخش پیاده‌سازی، باید دقت شود که خانه‌های ثابت به هیچ‌وجه در طول فرآیند الگوریتم تغییر نکنند. همچنین لازم است هر مرحله‌ی تولید، انتخاب، ترکیب و جهش به‌صورت مستقل و قابل کنترل طراحی شود تا در صورت نیاز امکان اصلاح و بهبود فراهم باشد. برای کنترل کیفیت خروجی، می‌توان در پایان هر نسل، بهترین فرد از لحاظ شایستگی را ذخیره کرده و نمودار پیشرفت شایستگی را ترسیم کرد. پیاده‌سازی باید به گونه‌ای باشد که الگوریتم پس از رسیدن به حد مشخصی از شایستگی یا تعداد نسل متوقف شود.

الگوریتم برای تعداد مشخصی از نسل‌ها تکرار می‌شود یا تا زمانی که یک جواب با شایستگی ۱۶۲ پیدا شود. در هر نسل، افراد بهتر حفظ می‌شوند (elitism) و باقی جمعیت از طریق ترکیب و جهش تولید می‌شود. در نهایت، جدولی که بیشترین شایستگی را دارد به عنوان بهترین جواب گزارش می‌شود.

این روش اگرچه تضمین نمی‌کند که همیشه جواب کامل به دست بیاید، اما در عمل می‌تواند جواب‌های بسیار نزدیک و یا کامل تولید کند و به عنوان یک روش هوشمند برای حل سودوکو مطرح شود.

سوال 4) پیاده‌سازی: مسئله فروشنده دوره‌گرد با استفاده از الگوریتم ژنتیک

در این تمرین، هدف شما طراحی و پیاده‌سازی یک الگوریتم ژنتیک برای حل مسئله کلاسیک فروشنده دوره‌گرد (Traveling Salesman Problem – TSP) است. در این مسئله، یک فروشنده باید از مجموعه‌ای از شهرها بازدید کرده و در نهایت به شهر آغازین بازگردد، به گونه‌ای که از هر شهر دقیقاً یک‌بار عبور کرده و مسیر پیموده‌شده، کمترین هزینه یا فاصله ممکن را داشته باشد.

داده‌های ورودی این مسئله به صورت یک ماتریس $n \times n$ در فایل به نام data.txt ارائه شده‌اند. خط اول این فایل تعداد شهرها را مشخص می‌کند و خط دوم عدد بزرگی را به عنوان مقدار مسیر ممنوع (یعنی سفر از هر شهر به خودش) ارائه می‌دهد. خطوط بعدی، فاصله یا هزینه سفر بین شهرها را نشان می‌دهند. به عنوان نمونه می‌توانید از فایل data.txt که حاوی اطلاعات مربوط به ۳۰ شهر است، برای تست الگوریتم خود استفاده کنید.

در الگوریتم ژنتیک این تمرین، هر فرد در جمعیت باید یک جایگشت از اعداد ۱ تا n باشد که ترتیب بازدید از شهرها را مشخص می‌کند. برای مثال، یک جایگشت مانند [3, 1, 5, 2, 4] نشان‌دهنده‌ی سفری از شهر ۴ به ۲، سپس ۵، سپس ۱، سپس ۳ و بازگشت به شهر ۴ است. فضای جستجوی این مسئله بسیار بزرگ است، بنابراین استفاده از تکنیک‌های تکاملی برای یافتن پاسخ تقریبی بهینه ضروری است.

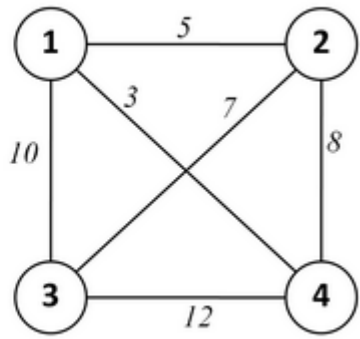
در پیاده‌سازی الگوریتم ژنتیک، برای انتخاب والدین باید از روش Tournament Selection با اندازه ۳ استفاده کنید. برای انتخاب بازماندگان جهت نسل بعد، باید از روش انتخاب $(\mu + \lambda)$ استفاده شود، به این معنا که والدین و فرزندان در یک مجموعه ادغام شده و بهترین افراد بر اساس تابع شایستگی انتخاب می‌شوند.

برای عملگر تقاطع، پیاده‌سازی روش Order Crossover الزامی است. این روش، یک بازه تصادفی از یک والد را انتخاب کرده و جایگزین بخش مربوطه در والد دوم می‌کند، به طوری که ساختار جایگشتی حفظ شود. برای جهش، باید از Inversion Mutation استفاده کنید؛ بدین صورت که یک زیربازه‌ی تصادفی از جایگشت معکوس می‌شود. تابع شایستگی در این مسئله، برابر است با مجموع هزینه سفر بین شهرهای متوالی در جایگشت ارائه‌شده، به علاوه هزینه بازگشت به شهر اول. هدف، کمینه‌سازی این مقدار است.

برنامه‌ی شما باید پس از اجرا، بهترین مسیر به دست آمده، مقدار کل هزینه‌ی آن، و همچنین نموداری از تغییرات مقدار شایستگی در طول نسل‌ها را نمایش دهد یا ذخیره کند. همچنین نتایج باید تحلیل شده و در یک گزارش کتبی ارائه شوند.

در گزارش خود، حتماً موارد زیر را درج کنید:

- میزان موفقیت یا عدم موفقیت الگوریتم در دستیابی به مسیر کوتاه
- تحلیل عملکرد الگوریتم در داده‌ی تست شده (فایل data.txt)
- توضیح دقیق ساختار الگوریتم (انتخاب، تقاطع، جهش، تابع شایستگی)
- نمودار یا جدول از تغییرات شایستگی در طول نسل‌ها
- اگر روش دومی برای عملگر تقاطع پیاده‌سازی کردید (مثل PMX یا CX)، نتایج آن را نیز مقایسه کنید.
- در ادامه مثالی از نوع ورودی آمده است که به شما در فهم سوال کمک می‌کند.

Line 1 - number of cities (n) Line 2 - value of 'no link' Next n lines -- c ₁₁ c ₁₂ ... c _{1n} c ₂₁ c ₂₂ ... c _{2n} ... c _{n1} c _{n2} ... c _{nn}	4 99999 99999 5 10 3 5 99999 7 8 10 7 99999 12 3 8 12 99999	
--	--	---

سوال 5) پیاده‌سازی: رمزگشایی متن رمزنگاری شده با استفاده از الگوریتم ژنتیک

در این تمرین، هدف شما طراحی و پیاده‌سازی یک الگوریتم ژنتیک برای حل یکی از مسائل کلاسیک رمزنگاری به نام کریپتوگرام تک‌حرفی (Single-letter Substitution Cryptogram) است. در این نوع رمزنگاری، که اغلب در کتاب‌ها و مجلات سرگرمی دیده می‌شود، هر حرف از الفبای زبان انگلیسی با یک حرف دیگر از همان الفبا به صورت یکتا جایگزین می‌شود (جایگشت کامل). حروف کوچک و بزرگ معادل در نظر گرفته می‌شوند و سایر کاراکترها مانند فاصله‌ها، نقطه‌گذاری‌ها و اعداد تغییر نمی‌کنند.

شما باید الگوریتمی طراحی کنید که با استفاده از تکنیک‌های تکاملی، یعنی الگوریتم ژنتیک، بتواند این جایگشت را کشف کرده و متن رمزنگاری شده را به متن اصلی و قابل خواندن بازگرداند. مسئله به صورت جستجو در فضای تمامی جایگشت‌های ممکن از حروف A تا Z در نظر گرفته می‌شود، و الگوریتم شما باید در بین این جایگشت‌ها، گزینه‌ای را پیدا کند که بهترین کیفیت رمزگشایی را ارائه دهد.

• تابع شایستگی

برای ارزیابی کیفیت هر راه‌حل (یعنی هر جایگشت)، باید یک تابع شایستگی تعریف شود که بتواند تخمین بزند آیا متن رمزگشایی شده حاصل از آن جایگشت، به زبان انگلیسی شباهت دارد یا خیر. یکی از ساده‌ترین و در عین حال مؤثرترین روش‌ها، مقایسه‌ی فراوانی وقوع حروف در متن رمزگشایی شده با فراوانی استاندارد آن‌ها در زبان انگلیسی است.

برای این منظور، لازم است یک منبع متنی بزرگ به زبان انگلیسی انتخاب و از آن برای استخراج فراوانی وقوع حروف A تا Z استفاده شود. جهت هماهنگی بین دانشجویان و مقایسه‌پذیر بودن نتایج، منبع مرجع باید کتاب "Pride and Prejudice" نوشته‌ی Jane Austen از سایت Project Gutenberg باشد:

<https://www.gutenberg.org/ebooks/1342>

پس از استخراج فراوانی استاندارد هر حرف (که آن را p_i برای حرف i می‌نامیم)، و محاسبه‌ی فراوانی واقعی حرف i در متن رمزگشایی شده با جایگشت فعلی (که آن را f_i می‌نامیم)، می‌توانید تابع شایستگی را به صورت زیر تعریف کنید:

$$F = (f_a - p_a)^2 + (f_b - p_b)^2 + \dots + (f_y - p_y)^2 + (f_z - p_z)^2$$

در این فرمول:

• f_i : درصد وقوع حرف i در متن رمزگشایی شده حاصل از جایگشت فعلی است.

• P_i : درصد وقوع حرف i در منبع مرجع (کتاب Jane Austen) است.

هدف الگوریتم ژنتیک این است که مقدار F را حداقل کند. هرچه مقدار F کمتر باشد، شباهت آماری بین متن رمزگشایی شده و زبان انگلیسی بیشتر است، و در نتیجه احتمال خوانا بودن آن بیشتر است.

در عمل، برای محاسبه‌ی فراوانی حروف:

1. متن رمزگشایی شده را از روی جایگشت فعلی تولید کنید.

2. تعداد تکرار هر حرف (A تا Z) را بشمارید.

3. نسبت هر حرف به کل حروف متنی (نه کاراکترها) را محاسبه کنید.

4. از آن‌ها به عنوان f_i در تابع شایستگی استفاده کنید.

توجه: اگر مایل باشید می‌توانید نسخه‌های پیشرفته‌تر تابع شایستگی را نیز به کار ببرید، مانند:

• استفاده از فراوانی دوحرفی‌ها (bigram) مانند IN, HE, TH, ...

• استفاده از الگوی شروع/پایان واژگان

• یا حتی بررسی کلمات واقعی با استفاده از لغت‌نامه

اما برای شروع، تابع فراوانی تک‌حرفی کفایت می‌کند و اجرای آن ساده و قابل پیاده‌سازی است.

• ورودی برنامه

برنامه‌ی شما باید یک فایل متنی رمزنگاری شده را به عنوان ورودی دریافت کند. در این فایل، فقط حروف الفبا جایگزین شده‌اند و سایر علائم از جمله فضا، نقطه‌گذاری، اعداد و حروف غیرانگلیسی بدون تغییر باقی مانده‌اند.

مثال یک ورودی رمزنگاری شده:

Gtunfwveqw: Mfdue fw gtd, mfdue fw zrtpr, twb xtue fw enr WtefqwtX Xrtvor Rtue

فایل های رمز موجود برای تست در دو فایل ew.txt و ega.txt می باشد که از آن باید استفاده کنید و در فایل تمرین آپلود شده است.

• خروجی برنامه

در پایان اجرای برنامه، شما باید دو خروجی تولید کنید:

1. output.txt

متنی که با جایگشت نهایی رمزگشایی شده است و باید تا حد ممکن قابل خواندن باشد.

برای مثال بالا خروجی به شرح زیر است:

Washington: First in war, first in peace, and last in the National League East

2. key.txt

نگاشت نهایی جایگشت بین حروف رمز و حروف اصلی، به صورت جدول مانند:

plaintext	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
ciphertext	t	h	p	b	r	m	v	n	f	i	y	x	j	w	q	z	s	d	u	e	o	l	g	a	k	c

در گزارش نهایی تمرین، باید موارد زیر را به طور کامل ارائه دهید:

- شرح ساختار کلی الگوریتم تکاملی طراحی شده (تولید جمعیت، انتخاب، تقاطع، جهش، توقف)
- توضیح دقیق تابع شایستگی و نحوه محاسبه ی آن
- فایل مرجع استفاده شده برای فراوانی حروف (کتاب Jane Austen) و نحوه محاسبه فراوانی ها
- بخشی از متن خروجی رمزگشایی شده نهایی
- جدول کلید جایگشت نهایی
- تحلیل عملکرد الگوریتم (مثلاً همگرایی، تعداد نسل ها، دقت، چالش ها)
- در صورت استفاده از روش های پیشرفته تر (مثل bigram)، حتماً آن را شرح دهید (امتیازی).

دقت داشته باشید که یک راه حل موفق در این تمرین، زمانی حاصل می شود که متن رمزگشایی شده به طور کامل با متن اصلی تطابق داشته باشد یا از نظر خوانایی و ساختار زبانی بسیار به آن نزدیک باشد. اما باید توجه کنید که جایگشت نهایی حروف (کلید رمزگشایی) الزاماً یکتا نیست.

علت این موضوع آن است که ممکن است برخی از حروف الفبا در متن رمزنگاری شده اصلاً استفاده نشده باشند، یا برعکس، در متن اصلی وجود نداشته باشند. در چنین شرایطی، چندین جایگشت مختلف ممکن است منجر به متنی یکسان یا بسیار مشابه با متن اصلی شوند، چرا که تغییر در نگاشت حروف بلااستفاده تأثیری در خروجی نهایی ندارد.

برای مثال، اگر در متن رمز، حروفی مانند A, C, H, I, J, K, L, S, Y اصلاً استفاده نشده باشند، و متن اصلی نیز فاقد حروفی مثل X, Z, B, J, M, Y, V, Q, K باشد، هرگونه تغییر در نگاشت این حروف، تغییری در نتیجه رمزگشایی نهایی ایجاد نخواهد کرد. در نتیجه، چند جایگشت مختلف می توانند متن را درست رمزگشایی کنند، حتی اگر دقیقاً با کلید اصلی تطابق نداشته باشند.

این موضوع باید هنگام ارزیابی خروجی و تحلیل نتایج در گزارش شما مورد توجه قرار گیرد.

موفق باشید